Java Collections Framework Cheat Sheet

Standard interfaces, core implementations, key methods, complexity & utility operations

JAVA 8+ / SE REFERENCE

Collection<E> Interface

Root of Hierarchy

`add(E e)` / `addAll(Collection<? extends E> c)` : Inserts item(s). Returns `true` if modified.

`remove(Object o)` / `removeAll(Collection<?> c)` : Removes matching item(s).

`removeIf(Predicate<? super E> filter)` : Removes elements matching condition.

`contains(Object o)` / `containsAll(Collection<?> c)` : Checks element presence.

`size()` / `isEmpty()` / `clear()` : Size queries and emptying operations.

`iterator()` : Returns an `Iterator<E>` for sequential traversal.

`toArray()` / `toArray(T[] a)` : Converts collection to array representation.

`stream()` / `parallelStream()` : Returns sequential or parallel `Stream<E>`.

List<E> Interface

Ordered & Positional Access

Impls: `ArrayList`, `LinkedList`, `Vector`, `Stack`

`get(int index)` / `set(int index, E element)` : Reads or replaces element at index.

`add(int index, E element)` / `remove(int index)` : Inserts/removes element at index.

`indexOf(Object o)` / `lastIndexOf(Object o)` : Finds first/last index (-1 if absent).

`subList(int fromIndex, int toIndex)` : Returns a view of portion between indices.

`listIterator()` / `listIterator(int index)` : Returns bidirectional `ListIterator`.

`sort(Comparator<? super E> c)` : Sorts the list in-place using comparator.

Queue<E> & Deque<E> Interfaces

Buffers / Stacks

Impls: `ArrayDeque`, `LinkedList`, `PriorityQueue`

Operation	Throws Exception	Returns Special Value
Insert (Tail)	`add(e)`	`offer(e)`
Remove (Head)	`remove()`	`poll()`
Examine (Head)	`element()`	`peek()`

`addFirst(e)` / `addLast(e)`, `offerFirst(e)` / `offerLast(e)` : Double-ended inserts.

`removeFirst()` / `removeLast()`, `pollFirst()` / `pollLast()` : Double-ended removes.

`getFirst()` / `getLast()`, `peekFirst()` / `peekLast()` : Double-ended reads.

`push(e)` / `pop()` : Stack LIFO equivalents (same as `addFirst` / `removeFirst`).

Collections Utility Class

java.util.Collections

`Collections.sort(list, comp)` : Sorts list in-place ($O(n \log n)$).

`Collections.binarySearch(list, key)` : Searches sorted list ($O(\log n)$).

`Collections.reverse(list)` / `shuffle(list)` : Reverses or randomizes list.

`Collections.min(coll)` / `max(coll)` : Finds extreme elements.

`Collections.unmodifiableList(list)` : Wraps collection as read-only view.

`Collections.synchronizedMap(map)` : Wraps collection with synchronized locks.

Set<E> & NavigableSet<E>

Unique Elements

Impls: `HashSet`, `LinkedHashSet`, `TreeSet`

Set<E> inherits Collection<E> methods, enforcing uniqueness.

`first()` / `last()` : Returns lowest or highest element in `TreeSet`.

`lower(e)` / `higher(e)` : Returns greatest element `< e` or smallest `> e`.

`floor(e)` / `ceiling(e)` : Returns greatest element `<= e` or smallest `>= e`.

`pollFirst()` / `pollLast()` : Retrieves and removes lowest or highest element.

`headSet(to)` / `tailSet(from)` / `subSet(from, to)` : Returns range views.

Map<K, V> Interface

Key-Value Pairs

Impls: `HashMap`, `LinkedHashMap`, `TreeMap`, `ConcurrentHashMap`

`put(K key, V val)` / `putIfAbsent(K key, V val)` : Inserts or updates mapping.

`get(Object key)` / `getOrDefault(key, defaultVal)` : Retrieves value.

`remove(key)` / `remove(key, value)` : Removes key mapping.

`containsKey(key)` / `containsValue(value)` : Key/Value membership tests.

`keySet()` : Returns `Set<K>` view of keys.

`values()` : Returns `Collection<V>` view of values.

`entrySet()` : Returns `Set<Map.Entry<K,V>>` set of pairs.

`computeIfAbsent(key, mappingFunction)` : Computes value if key missing.

`computeIfPresent(key, remappingFunction)` : Updates value if key present.

`merge(key, value, remappingFunction)` : Merges value with existing mapping.

Factory & Immutable Methods

Java 9+ / 10+

`List.of(e1, e2...)` : Returns unmodifiable list. No `null`s allowed.

`Set.of(e1, e2...)` : Returns unmodifiable set. Disallows duplicates/nulls.

`Map.of(k1, v1, k2, v2...)` : Returns unmodifiable map (up to 10 pairs).

`Map.ofEntries(entry(k, v)...)` : Unmodifiable map for arbitrary size.

`List.copyOf(coll)` / `Set.copyOf(coll)` : Creates unmodifiable copy.

Complexity & Property Matrix

Performance Guide

Class	Get / Search	Add / Remove	Nulls?	Key Traits
ArrayList	`O(1)` / `O(n)`	`O(1) amort.`	Yes	Fast index access, dynamic array.
LinkedList	`O(n)`	`O(1)` ends	Yes	Doubly-linked; high node overhead.
HashSet	`O(1)` avg	`O(1)` avg	Yes (1)	Unordered set backed by hash table.
TreeSet	`O(log n)`	`O(log n)`	No	Red-Black tree; sorted elements.
HashMap	`O(1)` avg	`O(1)` avg	Key: 1	Fast key-value lookup table.
TreeMap	`O(log n)`	`O(log n)`	No Keys	Sorted by keys; NavigableMap.
ArrayDeque	`O(n)` search	`O(1)` ends	No	Resizable array; better than Stack.